



Week 3 Solution

Question 1

Question

Which of the following statements is true about kernel and memory (RAM) in Linux and xv6?

1. Kernel resides in the upper part of the memory
2. Kernel resides in the lower part of the memory
3. Kernel does not reside completely in the memory
4. Kernel resides anywhere in memory and this is kept track by a universal page table

Answer

Kernel resides in the lower part of the memory

Solution

The kernel occupies a designated region of memory rather than being placed randomly.

In the memory layout considered in the assignment, the kernel is located in the lower part of memory.

The operating system also uses mechanisms such as page tables and hardware protection to prevent ordinary user programs from accessing protected kernel memory.

Therefore, the correct option is:

Option 2 — Kernel resides in the lower part of the memory.

Question 2

Question

State True or False:

There is a Kernel stack and a User stack in the process address space to ensure that attacks that target the stack will not affect the kernel.

Answer

True

Solution

A process has a user stack for executing user-level code. When the process enters kernel mode, such as during a system call or interrupt, the kernel uses a separate kernel stack.

Conceptually:



The separation between the user stack and kernel stack provides protection.

If a malicious user program corrupts its own user stack, it should not directly overwrite the kernel's stack.

Therefore, the statement is:

True.

Question 3

Question

A process after running for a while is preempted from the processor for other processes to run. Then its process state is moved to:

1. NEW
2. READY
3. RUNNING
4. BLOCK

Answer

READY

Solution

A process in the **RUNNING** state is currently using the CPU.

Suppose the scheduler decides to take the CPU away from this process so that another process can run. This is called **preemption**.

The process is still capable of running. It is simply waiting for the CPU to become available again.

Therefore:

RUNNING

|

| Preemption

↓

READY

It does not move to **BLOCKED** because it is not waiting for an I/O operation or some other event.

Hence, the correct answer is:

READY

Question 4

Question

What is the output in the child process?

```
char fg = '0';

int pid = fork();

if (pid != 0)
{
    printf("fg = %d", fg + 2);
}
else
{
    printf("fg = %d", fg - 2);
}
```

Options:

1. **fg = M**

2. `fg = 0`
3. `fg = 77`
4. `fg = 81`

Answer

`fg = 77`

Solution

After `fork()` , there are two processes:

- Parent
- Child

The return value of `fork()` is:

```
Parent → child's PID
Child  → 0
```

Therefore, the child executes the `else` block:

```
printf("fg = %d", fg - 2);
```

The character is:

`'0'`

Its ASCII value is:

`'0' = 79`

Therefore:

```
fg - 2
= 79 - 2
= 77
```

The format specifier is `%d` , so the numeric value is printed.

Thus:

```
fg = 77
```

Note that if `%c` had been used instead of `%d` , ASCII 77 would represent `M` .

Therefore, the correct answer is:

```
fg = 77
```

Question 5

Question

Which of the following are true about the `fork()` system call?

Multiple answers are correct.

1. All parent pages are initially marked as shared
2. `fork()` returns 0 to the parent process
3. `fork()` returns the parent's process ID to the child process
4. To execute another program in the child process, another system call `exec` is invoked in the child process after `fork`

Answer

Options 1 and 4

Solution

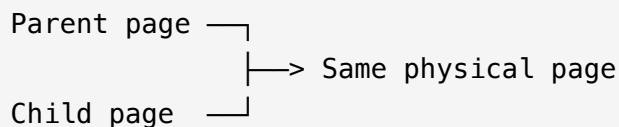
Option 1: All parent pages are initially marked as shared

True.

`fork()` can use **Copy-on-Write (COW)**.

Instead of immediately making a complete copy of the parent's memory, the parent and child can initially share the same physical pages.

Conceptually:



If one process tries to modify a shared page, the operating system can create a separate copy.

Therefore, option 1 is correct.

Option 2: `fork()` returns 0 to the parent

False.

The return value is:

```
Parent → child's PID
Child  → 0
```

So `0` is returned to the child, not the parent.

Option 3: `fork()` returns the parent's process ID to the child

False.

The child receives `0` from `fork()` .

Therefore, this statement is incorrect.

Option 4: `exec` can be invoked after `fork()`

True.

A common pattern is:

```
pid = fork();

if (pid == 0)
{
    exec(...);
}
```

`fork()` creates the child process, while `exec()` replaces the child's current program image with another program.

Therefore, options **1 and 4** are correct.

Question 6

Question

Which of the following methods can be used to terminate a process involuntarily?

1. `fork`
2. `kill`

3. `exec`
4. `exit`

Answer

`kill`

Solution

`exit()` is normally used by a process to terminate itself voluntarily.

On the other hand, `kill()` allows a process to send a signal to another process.

For example:

```
kill(pid, SIGKILL);
```

Here, the process identified by `pid` can be forcibly terminated.

The other options do not represent involuntary process termination:

- `fork()` → creates a new process
- `exec()` → replaces the program being executed
- `exit()` → terminates the calling process

Therefore:

Correct answer: `kill`

Question 7

Question

Which of the following are true about zombie entries?

1. A zombie entry is removed when its parent reads its exit status

2. When the process terminates, it becomes a zombie and is immediately removed
3. All zombies are a resource leak and will continue to exist infinitely
4. If a process terminates creating at least one intentional orphan, it does not become a zombie

Answer

A zombie entry is removed when its parent reads its exit status.

Solution

When a child process terminates, the operating system needs to retain information about the terminated process, including its exit status.

The child can therefore temporarily remain as a **zombie**.

The sequence is:

```
Child is running
    ↓
Child terminates
    ↓
Zombie entry remains
    ↓
Parent calls wait()
    ↓
Parent obtains exit status
    ↓
Zombie entry is removed
```

The `wait()` system call allows the parent to collect the child's termination status.

Therefore, once the parent reads the exit status, the zombie entry can be removed.

Hence, option 1 is correct.

Question 8

Question

State True or False:

When an `exit` system call gets executed, a wakeup signal is sent to its parent process because the parent may be sleeping due to a `wait`.

Answer

True

Solution

Consider the following situation:

```
Parent
|
| wait()
↓
Sleeping
```

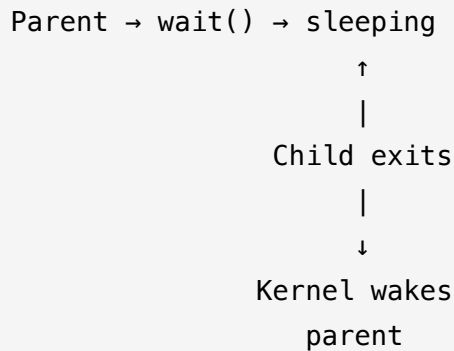
The parent may be waiting for one of its children to terminate.

Now suppose the child executes:

```
exit();
```

The kernel needs to notify/wake the parent because the event that the parent was waiting for has occurred.

Conceptually:



The parent can then continue and collect the child's exit status.

Therefore, the statement is:

True.

Question 9

Question

State True or False:

When the `wait` system call is invoked in the parent, it is going to be blocked until all of its child processes exit.

Answer

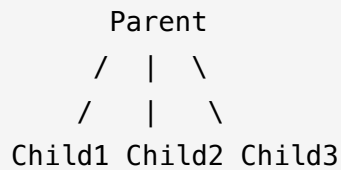
False

Solution

The key word in the statement is **"all."**

A `wait()` call does not necessarily wait for **all** child processes to terminate.

For example, suppose a parent has three children:



If one child terminates, a `wait()` operation can return information about a terminated child.

The parent does not have to wait for all three children to exit before a wait operation can complete.

If the parent needs to wait for multiple children, it can perform additional wait operations as appropriate.

Therefore, the statement is:

False.

Question 10

Question

Which of the following is NOT part of an ELF header?

1. Magic number
2. Type
3. Network details
4. Machine details

Answer

Network details

Solution

ELF stands for:

Executable and Linkable Format

An ELF header contains information that allows the operating system and other tools to identify and interpret the executable/object file.

For example, the ELF header contains:

Magic number

The beginning of an ELF file contains a magic number identifying it as an ELF file:

```
7F E L F
```

Type

The header contains information about the type of ELF file, such as whether it is an executable, relocatable file, shared object, etc.

Machine details

The ELF header contains information about the target machine/architecture.

However, **network details** are not a standard ELF-header field.

Therefore:

Correct answer: Network details.